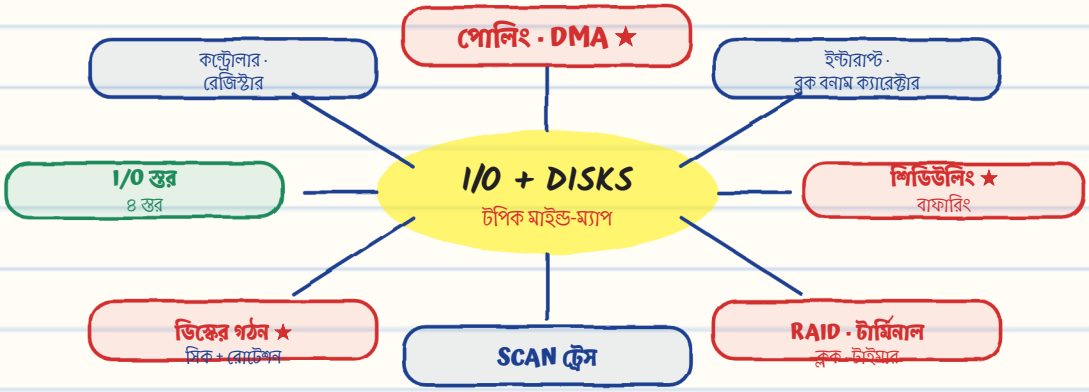
ইনপুট/আউটপুট, ডিস্ক ও ক্লক

কন্ট্রোলার · ইন্টারাপ্ট · DMA · ডিস্কের সময় ও শিডিউলিং · RAID · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = যে স্তর থেকে প্রথম প্রশ্নগুলো আসে

➜ সিলেবাসের যে অংশ

সিলেবাসে আছে "*Input/Output: hardware, software, disk, terminals, clocks*"। মেমোরি ম্যানেজমেন্ট মেমোরি লেসনে, ফাইল বরাদ্দ ফাইল সিস্টেম লেসনে।

হাই-ইল্ড তালিকা

① অ্যাক্সেস = সিক + ½ রোটেশন + ট্রান্সফার
② ½ রোটেশন = **30 ÷ rpm** সেকেন্ড
③ ট্রান্সফার = বাইট ÷ ট্রান্সফার রেট
④ CPU ছোঁয় শুধু কন্ট্রোলারের **রেজিস্টার**
⑤ পোর্ট-ম্যাপড = নিজস্ব IN/OUT ঠিকানা
⑥ DMA = প্রতি ব্লকে **একটি** ইন্টারাপ্ট
⑦ SSTF ভোকে · SCAN ন্যায্য · C-SCAN সমান
⑧ RAID 5 = (N − 1) ডিস্ক, ১টি বাদ যায়

সংকেত হলুদ = মূল শব্দ · লাল বৃত্ত = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

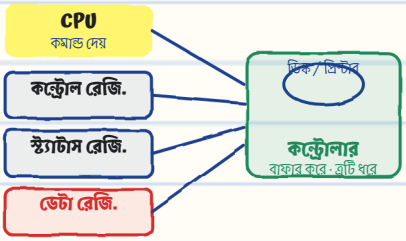
➜ সূচিপত্র

✓ I/O হার্ডওয়্যার ও রেজিস্টার ... পৃ. ২
✓ পোলিং, ইন্টারাপ্ট, DMA ... পৃ. ৩
✓ I/O সফটওয়্যারের স্তর ... পৃ. ৪
✓ শিডিউলিং ও বাফারিং ... পৃ. ৫
✓ ডিস্কের গঠন ও সময় ... পৃ. ৬
✓ ডিস্ক শিডিউলিং ট্রেস ... পৃ. ৭
✓ ফরম্যাটিং, বুট ব্লক, RAID ... পৃ. ৮
✓ টার্মিনাল ও ক্লক ... পৃ. ৯ · রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

এই পুরো লেসনের উদ্দেশ্য প্রতিটি I/O কাজ আসলে একটি দীর্ঘ অপেক্ষা — রেজিস্টারে মাইক্রোসেকেন্ড, ডিস্কে মিলিসেকেন্ড — এবং পুরো বিষয়টি হলো **ডিভাইস প্রস্তুত না হওয়া পর্যন্ত CPU-কে খাটিয়ে রাখা**র শিল্প।

১ I/O হার্ডওয়্যার: ডিভাইস, কন্ট্রোলার ও রেজিস্টার মূল বিষয় ★

সংজ্ঞা *I/O* বলতে কম্পিউটার ও বাইরের জগৎ — কীবোর্ড, প্রিন্টার, ডিস্ক, নেটওয়ার্ক — এর মধ্যে যেকোনো ডেটা স্থানান্তরকে বোঝায়। এ কাজ করে একটি **কন্ট্রোলার** (*I/O* সাবসিস্টেম, অ্যাডাপ্টার বা ইন্টারফেস কার্ড), যার তিনটি কাজ: এটি ডিভাইস ও মেইন মেমোরির মধ্যে ডেটা নিয়ে যায়, ডিভাইসের গতি লুকিয়ে রাখে, এবং ট্রান্সফার চলাকালীন *CPU*-কে **অন্য কাজ করতে দেয়**।



একটি ডিভাইসের প্রতি *CPU*-র দৃষ্টিভঙ্গি হলো **তিনটি রেজিস্টার** আর একটি ইন্টারাপ্ট লাইন

অংশ	আসলে কী
ডিভাইস	আসল কাজ করে এমন ভৌত একক: হেড, ড্রাম, প্রিন্ট হেড, কী
কন্ট্রোলার	মধ্যস্থামী — সাধারণ কমান্ড নেয়, বাফার করে, ত্রুটি জানায়
কন্ট্রোল রেজিস্টার	পরের কাজ কী (পড়া, লেখা, সিলিন্ডারে যাওয়া)
স্ট্যাটাস রেজিস্টার	কেমন চলছে: বিজি, রেডি, এরর — *CPU* এটাই পড়ে সিদ্ধান্ত নেয়
ডেটা রেজিস্টার	এক শব্দের বাফার, যেখান দিয়ে বাইট *CPU* ও ডিভাইসের মধ্যে যায়

ডিভাইস অ্যাড্রেসিং	পোর্ট-ম্যাপড (আইসোলেটেড) *I/O*	মেমোরি-ম্যাপড *I/O*
রেজিস্টার কোথায়	আলাদা পোর্ট ঠিকানা স্পেসে — *x86*-এ 2^{16} = 64 K পোর্ট	সাধারণ **মেমোরি ঠিকানায়** স্বাভাবিক অ্যাড্রেস স্পেসের ভেতরে
কোন ইনস্ট্রাকশন	শুধু বিশেষ *IN / OUT* — অল্প অ্যাড্রেসিং মোড	সাধারণ **লোড / স্টোর**; সব অ্যাড্রেসিং মোড কাজ করে
খরচ কোথায়	আলাদা ইনস্ট্রাকশন সেট; মেমোরি নষ্ট হয় না	মেমোরি অ্যাড্রেস স্পেস খায়, আর ওগুলো কখনো ক্যাশ করা যাবে না

ইন্টারাপ্ট **ইন্টারাপ্ট** হলো ডিভাইসের *CPU*-কে ডাকার পদ্ধতি — উল্টোটা নয়। তিন প্রকার: **ট্র্যাপ** (ভেতরের এরর — শূন্যে ভাগ, পেজ ফল্ট), **ইন্টারাপ্ট** (ডিভাইস থেকে, অ্যাসিঙ্ক্রোনাস), **সফটওয়্যার ইন্টারাপ্ট** (সিস্টেম কল)। প্রতিটি হ্যান্ডলারের ঠিকানা *CPU* একটি **ভেক্টর টেবিলে** রাখে; একসঙ্গে কয়েকটি আসলে **প্রায়োরিটি** ঠিক করে কে আগে চলবে।

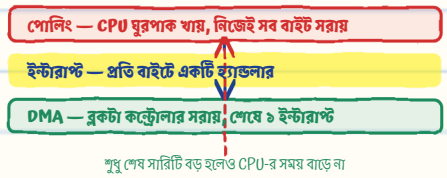
পরীক্ষার ফাঁদ *(i)* মেমোরি-ম্যাপড *I/O* ডিভাইসকে মেমোরি **বানায় না** — ঠিকানাটা এখনো কন্ট্রোলারের রেজিস্টারেই পড়ে। *(ii)* ছোট ইনস্ট্রাকশন সেট কারোটা বলে ওটা **পোর্ট-ম্যাপড** *I/O*; অ্যাড্রেস স্পেস খায় সেটা **মেমোরি-ম্যাপড**। *(iii)* ইন্টারাপ্ট **অ্যাসিঙ্ক্রোনাস**; ট্র্যাপ ঘটে চলতে থাকা ইনস্ট্রাকশন থেকেই।

মনে রাখার কৌশল **"তিনটি রেজিস্টার, একটি টেবিল।"** কন্ট্রোল বলে *কী*, স্ট্যাটাস বলে *কখন*, ডেটা বলে *কোন বাইট*; ভেক্টর টেবিল বলে *কোথায় লাফ দিতে হবে*। এই লেসনের বাকি সবই হলো কন্ট্রোল থেকে স্ট্যাটাস পর্যন্ত অপেক্ষা এড়ানোর একটি কৌশল। আধুনিক মেশিনে একটি *PCIe/USB/SATA* কন্ট্রোলার চিপের পেছনে সব একই থাকে — *CPU* তার কাছে যায় **মেমোরি-ম্যাপড** রেজিস্টার ও *MSI-X* ইন্টারাপ্ট দিয়ে।

২ পোলিং, ইন্টারাপ্ট-চালিত I/O ও ডিএমএ

সবচেয়ে বেশি আসে ★

সংজ্ঞা এই পাতার প্রশ্নটা সবসময় একটাই: বাইগুলো কে বহন করবে, আর তার সময় খরচ কে দেবে? **পোলিং**-এ CPU বহন করে এবং পুরো অপেক্ষার খরচও দেয়; **ইন্টারাপ্ট-চালিত I/O**-তে প্রতিবার এক বাইট নিয়ে CPU বহন করে ও প্রতিবার একটি হ্যান্ডলারের খরচ দেয়; **DMA**-তে **কন্ট্রোলার** পুরো ব্লক বহন করে আর CPU শুধু **শেষে একবার** খরচ দেয়।



একই 1 KB ব্লক, একই ডিভাইস — বদলায় শুধু CPU-র অংশ

পদ্ধতি	ডেটা কে সরায়	ইন্টারাপ্ট	মূল্যায়ন
পোলিং	CPU, এক শব্দ করে	কোনোটি নয়	CPU ঘুরপাক করে
ইন্টারাপ্ট-চালিত	প্রতিবার ইন্টারাপ্টে CPU	১ / বাইট	CPU বাকি সময় কাজ করে; **~1 MB/s-এর নিচে পোলিং-এর চেয়ে খারাপ**
DMA ★	কন্ট্রোলার, পুরো ব্লক	**১ / ব্লক**	ডিস্ক বা নেটওয়ার্ক কার্ডের জন্য একমাত্র উপযুক্ত

ব্লক বনাম ক্যারেক্টার ডিভাইস একটি **ব্লক ডিভাইস** (ডিস্ক, SSD) ডেটা রাখে নির্দিষ্ট আকারের ব্লকে এবং তা **যেকোনো ক্রমে** পৌঁছানো যায়, তাই বাফারিং, ক্যাশিং ও I/O শিডিউলিং সবই কাজে লাগে। একটি **ক্যারেক্টার ডিভাইস** (কীবোর্ড, মাউস, টার্মিনাল, প্রিন্টার) একটি **ধারাবাহিক বাইট স্ট্রিম** দেয় — সেখানে সিক নেই, ব্লক নেই, তাই শিডিউল বা ক্যাশ করার কিছু নেই। দুটিই OS-এ ফাইল হিসেবে দেখা যায়।

DMA বাস ভাগ করে নেয় যেভাবে — আর দুই ধরনের ডিভাইস

সাইকেল স্টিলিং — একবারে এক মেমোরি সাইকেল নিয়ে বাস ফেরত দেয়; CPU ধীর হয় কিন্তু **থামে না**

ট্রান্সপারেন্ট মোড — CPU ব্যবহার না করলেই বাস নেয়; চাকরি চললে CPU **থামে**

বার্স্ট মোড — কন্ট্রোলারের নিজস্ব FIFO থেকে পুরো ব্লকের জন্য বাস ধরে রাখে; **সবচেয়ে বেশি থ্রুপুট**

সমাধানকৃত উদাহরণ ✍ 1 KB আসলে কত খরচ?

একটি ডিভাইস **প্রতি 1 µs-এ একটি বাইট** (1 MB/s) দেয়; ব্লক **1 KB**; একটি হ্যান্ডলারে CPU-র খরচ **2 µs**। **(ক)** ট্রান্সফারে সময় লাগে 1024 µs। **(খ) পোলিং:** CPU পুরো 1024 µs ব্যস্ত থাকে বাইট সরাতে। **(গ) ইন্টারাপ্ট-চালিত:** 1024 × 2 µs = **2048 µs CPU** — ট্রান্সফারের **দ্বিগুণ**, তাই এই গতিতে **পোলিং করলে দ্রুত**। **(ঘ) DMA:** ১টি ইন্টারাপ্ট = 1024 বাইটের জন্য 2 µs CPU, অর্থাৎ **0.2 %** ওভারহেড।

পরীক্ষার ফাঁদ (i) "ইন্টারাপ্ট-চালিত সবসময় পোলিং-এর চেয়ে ভালো" এটি **মিথ্যা**: কয়েকটি ইনস্ট্রাকশনের চেয়ে একটি হ্যান্ডলার বেশি CPU সময় নেয়। (ii) DMA-তেও CPU-কে ট্রান্সফার **সাজাতে** হয় — CPU সরে যায় না, শুধু বাইট ধরে বদলানোর কাজটুকু যায়।

মনে রাখার কৌশল "**ধীরে ঘোরো, দ্রুতে ইন্টারাপ্ট, আরও দ্রুতে DMA।**" আর "**সাইকেল চুরি, বার্স্ট আঁকড়ে, চুপচাপ অপেক্ষা।**" — সাইকেল স্টিলিং, বার্স্ট ও ট্রান্সপারেন্ট তিন বাক্যে।

আইটি সংযোগ মাইক্রোপ্রসেসর লেসনের 8237 DMA কন্ট্রোলার ঠিক এই কাজটিই করে — চারটি চ্যানেল, প্রতিটি দিয়ে একটি করে ডিভাইস।

৩ I/O সফটওয়্যারের চারটি স্তর লিখিত প্রশ্নের প্রিয় ★

সংজ্ঞা *I/O* সফটওয়্যার চারটি স্তরে তৈরি, আর পুরো নকশাটাই একটি প্রশ্নের উত্তর: **হার্ডওয়্যার-নির্দিষ্ট জ্ঞান কতদূর নামে যায়?** কন্ট্রোলারের ঠিক উপরের স্তরটিই মাত্র জানে ডিভাইসটি আছে, তাই নতুন ডিস্ক মানে **একটি নতুন ড্রাইভার** — নতুন প্রোগ্রাম নয়।

১ · ইউজার-লেভেল I/O লাইব্রেরি
২ · ডিভাইস-ইন্ডিপেন্ডেন্ট সফটওয়্যার
৩ · ডিভাইস ড্রাইভার
৪ · ইন্টারাপ্ট হ্যান্ডলার

স্তরগুলো, `read()` কল করা প্রোগ্রাম থেকে ডিভাইস পর্যন্ত

স্তর	কী করে
১ · ইউজার-লেভেল *I/O* সফটওয়্যার	প্রোগ্রাম যে লাইব্রেরি বলে; আউটপুট লাইনে বাফার করে, ফরম্যাট করে, ইউজার ও কার্নেল স্পেসের মধ্যে নেয়
২ · ডিভাইস-ইন্ডিপেন্ডেন্ট স্তর	সমান নাম ও প্রোটেকশন, ডিভাইস বরাদ্দ ও ফেরত, **বাফারিং**, ব্লকের আকার, ত্রুটির রিপোর্ট
৩ · ডিভাইস ড্রাইভার ★	**একমাত্র** হার্ডওয়্যার-সচেতন স্তর: সাধারণ অনুরোধকে কন্ট্রোলার কমান্ডে বদলায়, *I/O* শুরু করে, স্ট্যাটাস রেজিস্টার পড়ে
৪ · ইন্টারাপ্ট হ্যান্ডলার	ইন্টারাপ্টের অপেক্ষায় থাকে, কনটেক্সট সংরক্ষণ ও ফেরত করে, ডিভাইস শনাক্ত করে, ডেটা সরায়, অপেক্ষায় থাকা প্রসেস জাগায়

পরীক্ষার ফাঁদ *(i)* বাফারিং ও ত্রুটি ব্যবস্থাপনা **ডিভাইস-ইন্ডিপেন্ডেন্ট** স্তরে, ড্রাইভারে নয়। *(ii)* **ড্রাইভার** কার্নেল কোড; **ইন্টারাপ্ট হ্যান্ডলার** স্তরের **সবচেয়ে নিচের** স্তর, প্রতি ইন্টারাপ্টে কার্নেল মোডে চলে।

কে কীর উত্তর দেয়

ডিভাইসের নাম কে দেয়? স্তর ২ — একটি টেবিল লজিক্যাল নামকে ফিজিক্যাল নামে মেলায়

ডিভাইস কে চেনে? স্তর ৩, ড্রাইভার — আর কেউ নয়

ইন্টারাপ্টের অপেক্ষা কে করে? স্তর ৪, হ্যান্ডলার

***300* বাইটের লেখাকে পূর্ণ ব্লকে কে প্যাড করে?** স্তর ২

কার্নেলের ভেতরের ড্রাইভার — লিনাক্স — একটি খারাপ ড্রাইভার পুরো মেশিন নষ্ট করতে পারে, কারণ সেটি কার্নেলের ঠিকানাতেই চলে। **ইউজার মোডের ড্রাইভার** *(আধুনিক GPU ও USB স্ট্যাক)* ক্ষতি সীমাবদ্ধ রাখে: ড্রাইভার মরলে শুধু সেই ডিভাইসটি হারায়, মেশিন চালু থাকে; দাম হলো একটি বার্তা পাঠানো-পাওয়া।

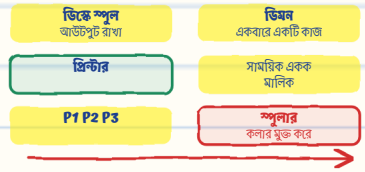
মনে রাখার কৌশল **"লাইব্রেরি, সাধারণ, ড্রাইভার, হ্যান্ডলার।"** স্তরগুলো পড়ুন *"কে চেয়েছে, কী নিয়ম, হার্ডওয়্যার কীভাবে চলে, কে জাগিয়েছে"* হিসেবে। লিখিত প্রশ্নে প্রায় সবসময় এই চারটি এই ক্রমে চাইয়া যায় — "বাফারিং কোন স্তরে?" জিজ্ঞেস করলে উত্তর **দ্বিতীয়**, "ড্রাইভার কী জানে?" জিজ্ঞেস করলে উত্তর **তৃতীয়**।

আইটি সংযোগ নতুন ডিভাইসের জন্য মডিউল লোড করা — *USB* প্রিন্টার, গ্রাফিক্স কার্ড — মানে রান টাইমে ঢোকানো স্তর ৩, আর এর উপরের কিছুই বদলায় না। তাই একই প্রোগ্রাম যেকোনো ডিস্কের সঙ্গে কাজ করে।

৪ I/O শিডিউলিং, বাফারিং, ক্যাশিং ও স্পুলিং হাই-ইল্ড ★

সংজ্ঞা একটি ডিভাইস একসঙ্গে একটিই অনুরোধ সামলাতে পারে, তাই সেগুলো জমে থাকে একটি **ডিভাইস কিউ**-তে। **I/O শিডিউলিং** হলো পরেরটি কোনটি সামলানোর নীতি, আর এর উদ্দেশ্য হলো কোনো অনুরোধকে **ভোকা** না দেওয়া।

I/O শিডিউলিং নীতি	কী সামলায়	মূল্যায়ন
FCFS / ফেয়ার-শেয়ার	আসার ক্রমে, প্রতি প্রসেসকে একবার করে	ন্যায্য; তবে একটি বড় সিক পিছনের সবকে আটকে দেয়
প্রায়োরিটি	সর্বোচ্চ প্রায়োরিটির অনুরোধ আগে	কম প্রায়োরিটি ভোকা খেতে পারে
SSTF (সর্বনিম্ন সিক আগে)	এখনকার অবস্থানের সবচেয়ে **কাছের** অনুরোধ	গড় অপেক্ষা সবচেয়ে কম, কিন্তু দূরের অনুরোধ ভোকা খায়
EDF (নিকটতম ডেডলাইন আগে)	ডেডলাইন সবচেয়ে কাছের অনুরোধ	সবচেয়ে ন্যায্য নীতি; রিয়েল-টাইম সেবায়
র‍্যান্ডম (নো-সিক)	যেকোনো ক্রমে	ডিস্কে খারাপ; **SSD-তে সেরা**, কারণ প্রতিটি ব্লক সমান দূরে



স্পুলিং প্রিন্টার দ্রুত করে না – সে কখনো **খালি রাখে না**

বাফারিং: তিন পদ্ধতি ও তার খরচ

সিঙ্গেল: ভরাট হয়, তারপর খালি হয় — দুই পাশ আলাদা

ডাবল: একটি ভরাতে থাকে, অন্যটি খালি হতে থাকে

সার্কুলার: একটি বলয়; ব্যবহৃত বাফার **আবার ব্যবহার** হয়

C = গণনার সময়, M = একটি ব্লক স্থানান্তরের সময়

$T = C + M$ বাফার ছাড়া · $\max(C,M) + M$ সিঙ্গেল · $\max(C,M)$ ডাবল · nটি সার্কুলার বাফারে $\max(C, M \div n) + M$।

ক্যাশিং বনাম স্পুলিং **ক্যাশিং** এমন একটি অনুলিপি রাখে যা সহজেই আবার আনা যায়, তাই এটি পড়া দ্রুত করে। **স্পুলিং** কোনো প্রসেসের **পুরো আউটপুট** ডিস্কে রাখে যাতে একান্ত ব্যবহৃত ডিভাইস ভাগ করা যায়। ক্যাশ একই পাঠককে আবার দেয়, স্পুলিং অনেক প্রসেসকে দেয়।

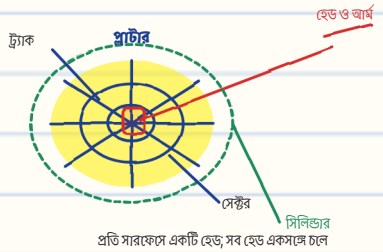
সমাধানকৃত উদাহরণ ✍ প্রিন্টারের কেস স্টাডি একটি লেজার প্রিন্টার **প্রতি 10 s-এ ১ পৃষ্ঠা** করে (6 পৃষ্ঠা/মিনিট)। একটি প্রোগ্রাম 500 পৃষ্ঠার রিপোর্ট বানাতে প্রতি পৃষ্ঠায় **0.2 s** কাজ লাগে। **(ক)** লেখা তৈরি = $500 \times 0.2 = 100$ s; ছাপা = 500×10 s = 5000 s ≈ 83.3 মিনিট, তাই বাধা প্রিন্টারটি। **(খ) বাফার নয়:** $10 + 0.2 = 10.2$ s প্রতি পৃষ্ঠা। **সিঙ্গেল:** $\max(0.2, 10) + 10 = 20$ s প্রতি পৃষ্ঠা ⇒ 500 পৃষ্ঠায় 10 000 s, অর্থাৎ **বাফার না রাখলেও খারাপ**। **ডাবল:** $\max(0.2, 10) = 10$ s প্রতি পৃষ্ঠা = 5000 s, কেবল ছাপার সময়। **(গ) স্পুলিং** হলে প্রোগ্রাম 100 s পরেই ফিরে যায় আর একটি ডিমন ধীরে ছাপে, ততক্ষণে আরও দশটি প্রোগ্রাম নিজের আউটপুট সারিবদ্ধ রাখতে পারে।

পরীক্ষার ফাঁদ *(i)* **SSTF হলো I/O শিডিউলিং, SJF হলো CPU শিডিউলিং** — দুটোই "সবচেয়ে ছোটটা আগে", কিন্তু ভিন্ন রিসোর্সের জন্য। *(ii)* ডাবল বাফারিং **গণনা ও ট্রান্সফার** একসঙ্গে চালায়; কোনোটাই দ্রুত করে না। *(iii)* ক্যাশ একই ডেটা আবার দেয়, স্পুলিং ভিন্ন প্রসেসকে দেয়।

মনে রাখার কৌশল "**বাফার = ফাঁক বাঁধে, স্পুল = একান্ত ডিভাইস ভাগ করে, ক্যাশ = পুনরায় পড়া এড়ায়।**" সার্কুলার বাফার আসলে "আরও স্লটসহ ডাবল বাফার"।

৫ ডিস্কের গঠন ও অ্যাক্সেস সময় হিসাব ★

সংজ্ঞা ম্যাগনেটিক ডিস্ক হলো একটি স্পিন্ডলে গাঁথা **প্লাটারের** স্তূপ, নির্দিষ্ট গতিতে ঘুরছে। ডেটা থাকে কেন্দ্রীয় **ট্র্যাকে**; প্রতিটি ট্র্যাক নির্দিষ্ট দৈর্ঘ্যের **সেক্টরে** কাটা, আর সেক্টরই সবচেয়ে ছোট স্থানান্তরযোগ্য একক; প্রতিটি সারফেসে একটি **রিড/রাইট হেড** থাকে; আর **প্রতিটি** সারফেসের একই ব্যাসার্ধের ট্র্যাকগুলো মিলে একটি **সিলিন্ডার** — যা হেড না সরিয়েই পড়া যায়।



সিলিন্ডার কোনো আকৃতি নয় – একই ব্যাসার্ধের সব ট্র্যাকের সমষ্টি

পরিভাষা	অর্থ
প্লাটার / সারফেস	একটি চুম্বকযোগ্য ডিস্ক, এক পাশ ব্যবহারযোগ্য ⇒ **হেডের সংখ্যা = সারফেস**
ট্র্যাক	এক সারফেসের একটি বৃত্ত; **সারফেসপ্রতি ট্র্যাক = সিলিন্ডার**
সেক্টর	স্থানান্তরের একক — **512 B**, অথবা অ্যাডভান্সড ফরম্যাটে **4 KiB**
ঠিকানা	**CHS** = (সিলিন্ডার, হেড, সেক্টর); আধুনিক ডিস্ক **LBA** দেয়

পরীক্ষার ফাঁদ *(i)* পুরোনো ডিস্কে প্রতিটি ট্র্যাকে **সমান সেক্টর** থাকে; আধুনিকে **জোন বিট রেকর্ডিং** হয়, তাই বাইরের ট্র্যাকে **বেশি** সেক্টর। *(ii)* সিলিন্ডার একটি **ট্র্যাকের সমষ্টি**, কোনো আকৃতি নয়।

তিনটি সময় ও সেই একটি সূত্র যা দিয়ে প্রশ্ন হয়

সিক টাইম T_s — হেড ট্র্যাকে নিয়ে যাওয়া

রোটেশনাল ডিলে $T_r = ½ \times 60 \div rpm$ সে

ট্রান্সফার টাইম T_t = বাইট ÷ ট্রান্সফার রেট

অ্যাক্সেস টাইম = $T_s + T_r + T_t$

যেহেতু **একটি সেক্টর এক ঘূর্ণনের যেকোনো বিন্দুতে এসে পৌঁছায়, তাই গড় রোটেশনাল ডিলেই এক ঘূর্ণনের অর্ধেক** — অর্থাৎ $T_r = 30 \div rpm$ সেকেন্ড, আর হিসাবের সবচেয়ে ছোট উপায় এটাই।

সমাধানকৃত উদাহরণ ✍ গড় অ্যাক্সেস সময় একটি ডিস্ক **7200 rpm**-তে ঘোরে, গড় সিক টাইম **4 ms**, ট্রান্সফার রেট **200 MB/s**, আর পড়া হয় একটি **4 KiB** ব্লক। **(ক)** এক ঘূর্ণন = $60 \div 7200$ = **8.33 ms**, তাই গড় রোটেশনাল ডিলে = **4.17 ms**। **(খ)** ট্রান্সফার = $4096\ B \div (200 \times 10^6\ B/s) = 2.05 \times 10^{-5}\ s = 0.02$ **ms**। **(গ)** গড় অ্যাক্সেস = $4 + 4.17 + 0.02 = 8.19$ **ms**। **(ঘ)** রোটেশনাল ডিলে $4.17 \div 8.19 \approx$ মোটের **51 %** — প্রতিটি অ্যাক্সেসে অর্ধেক সময় কেবল সেক্টরের অপেক্ষায় যায়। **(ঙ)** 14400 rpm-এ ডিলেই অর্ধেক হয়ে **2.08 ms**, মোট **6.10 ms**; ট্রান্সফার রেট দ্বিগুণ করলে মাত্র **0.01 ms** বাঁচে।

মনে রাখার কৌশল "**সিক, স্পিন, শিপ।**" হেড সরানো, সেক্টরের অপেক্ষা, বাইট সরানো — এই ক্রমেই, আর সবসময় **যোগ** করতে হয়। "**30 ভাগ rpm**"-ই অর্ধেক ঘূর্ণনের অপেক্ষা সেকেন্ডে।

বাস্তবে ড্রাইভের নিজস্ব ইলেকট্রনিক্স এখন বেশিরভাগ সিক ঢেকে দেয়: এটি অনুরোধ নিজে সাজায় ও ছোট ক্যাশ রাখে, তাই OS যা দেখে তা মোটামুটি **8–10 ms**, হেড আসলে যতটা সরেছে তার উপর নির্ভর করে না।

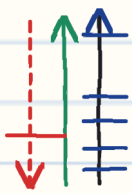
৬ ডিস্ক শিডিউলিং: FCFS, SSTF, SCAN, C-SCAN, LOOK ট্রেস করো ★

সংজ্ঞা অনুরোধ জমা হয় **সিলিন্ডার নম্বর** হিসেবে, আর ডিস্ক শিডিউলিং ঠিক করে হেড কোন ক্রমে সেগুলোর কাছে যাবে। মাপকাটি হলো মোট হেড চলাচল সিলিন্ডারে, কারণ প্রতিটি পেরোনো সিলিন্ডারের দাম একটি সিক। **SSTF** মোট চলাচল সবচেয়ে কমায় কিন্তু ভোকায়; **SCAN** প্রত্যেক অনুরোধকে **সীমিত অপেক্ষা** দেয় — লিখিত উত্তরে এই দোয়োটার দোটাই তুলনা করতে হয়।

অ্যালগরিদম	কোন ক্রমে সামলায়	মূল্যায়ন
FCFS	আসার ক্রমে	ভোকা হয় না, তবে মোট চলাচল সবচেয়ে বেশি
SSTF	যে সিলিন্ডার **সবচেয়ে কাছে**	**মোট সবচেয়ে কম** — দূরের অনুরোধ সারাবার ওদের দিকে আসতে পারে
SCAN (এলিভেটর)	একদিকে ডিস্কের **শেষ পর্যন্ত**, তারপর উল্টে	ভারসাম্যপূর্ণ প্রত্যুত্তর
C-SCAN	শুধু একদিকে; ফেরার পথে **কাউকে সামলায় না**	**অপেক্ষা সবচেয়ে সমান** — লিফট নয়, সারির মতো
LOOK / C-LOOK	SCAN, তবে ডিস্কের কিনারা নয়, **শেষ অনুরোধে** থেমে	বাস্তবসম্মত সংস্করণ; অপচয় কম

সমাধানকৃত ট্রেস ✍ হেড সিলিন্ডার 53-এ, আসার ক্রমে সারিঃ 98, 183, 37, 122, 14, 124, 65, 67; ডিস্ক 0–199

অ্যালগরিদম	হেড চলাচল, ধাপে ধাপে	মোট
FCFS	53→98→183→37→122→14→124→65→67 : 45+85+146+85+108+110+59+2	640
SSTF	53→65→67→37→14→98→122→124→183 : 12+2+30+23+84+24+2+59	236
SCAN ↑	53→65→67→98→122→124→183→199, তারপর নিচে 37→14 : 130+16+162+23	331
LOOK ↑	SCAN-এর মতো, কিন্তু 199-এ নয় 183-এ ফিরে : 130+169	299
C-SCAN ↑	199 পর্যন্ত উপরে (146), 0-এ লাফ (199), তারপর 0→14→37 (14+23)	382
C-LOOK ↑	183 পর্যন্ত উপরে (130), 14-এ ফিরে (169), তারপর 124 পর্যন্ত (110)	409



সিলিন্ডার 199 — ডিস্কের কিনারা

উপরে: লিফট পথেই সবাইকে সামলায়

নিচে: C-SCAN কাউকে সামলায় না

হেড শুরু করে 53-এ; SCAN নিচের পথেও সামলায়

অপেক্ষমাণ: 14 · 37 · 65 · 67 · 98 · 122 · 124 · 183

কোনো অনুরোধ একটি পূর্ণ স্ক্যানের বেশি অপেক্ষা করে না

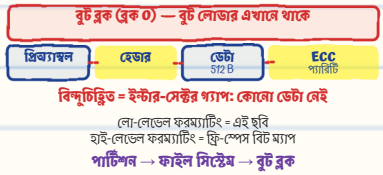
এলিভেটর পরিবার: প্রতিটি অনুরোধ একটি পূর্ণ স্ক্যানের মধ্যেই পৌঁছে যায়

পরীক্ষার ফাঁদ (i) SCAN ডিস্কের **শেষে** পৌঁছায়; LOOK শেষ অনুরোধে থামে — পুরো পার্থক্যই 331 − 32 = 299। (ii) C-SCAN শুধু একদিকে সামলায়। (iii) "সবচেয়ে কম মোট চলাচল" আর "সবচেয়ে কম গড় অপেক্ষা" **এক প্রশ্ন নয়**।

মনে রাখার কৌশল "**SSTF হলো কাছের, SCAN হলো লিফট, C-SCAN হলো সারি।**" লিফট উপরে-নিচে দুই দিকেই মানুষ নেয়; সারি একদিকে পরিবেশনায়, ফেরার পথে কেউ নেই।

৭ ফরম্যাটিং, বুট ব্লক, RAID ও সলিড-স্টেট ডিস্ক হিসাব ★

সংজ্ঞা ফাঁকা ডিস্কে কিছু রাখার আগে তা চিহ্নিত করতে হয়, আর তা হয় **দুই স্তরে**। **লো-লেভেল (ফিজিক্যাল) ফরম্যাটিং** পৃষ্ঠকে সেক্টরে ভাগ করে এবং প্রিঅ্যাম্বল, হেডার, ECC ও ইন্টার-সেক্টর গ্যাপ লেখে; এটি কারখানার কাজ। **হাই-লেভেল (লজিক্যাল) ফরম্যাটিং** লেখে **ফ্রি-স্পেস স্ট্রাকচার** — প্রতিটি ব্লক চিহ্নিত করা বিট ম্যাপ।



ফরম্যাট করা ডিস্ক কেন আর তার নামানো ধারযত্যের চেয়ে কম দেখায়

ফাঁকা ডিস্ক থেকে বুটযোগ্য সিস্টেম

১ · **লো-লেভেল ফরম্যাট:** সেক্টর, গ্যাপ, ECC

২ · **পার্টিশন টেবিল:** MBR (৪টি প্রাইমারি, 2 TB) বা GPT (১২৮, অনেক বেশি)

৩ · **হাই-লেভেল ফরম্যাট:** একটি পার্টিশনে ফাইল সিস্টেম

৪ · **বুট ব্লক:** ব্লক 0-এ বুট লোডার

BIOS/UEFI ব্লক 0 পড়ে বুট লোডার চালায়, সে কার্নেল লোড করে। **ব্যাড ব্লক স্পেয়ারিং** দিয়ে সামলানো হয়: বিষয়বস্তু একটি রিজার্ভ সেক্টরে সরে যায় আর নম্বর নতুন করে বসে।

পরীক্ষার ফাঁদ (i) সেক্টরগুলোর মধ্যে ইন্টার-সেক্টর গ্যাপ থাকে, তাই **ফরম্যাট করা ডিস্ক কখনো পূর্ণ নামানো ধারযত্য দেখায় না**। (ii) লো-লেভেল ফরম্যাটিং **কারখানার** কাজ; ব্যাড ব্লক **স্পেয়ার** হয়, ফরম্যাট করে মুছে ফেলা হয় না।

RAID স্তর	কীভাবে কাজ করে	ব্যবহারযোগ্য আয়তন (সর্বনিম্ন ডিস্ক)	কতটি ডিস্ক বাদ যেতে পারে
0 — স্ট্রাইপিং	ডেটা সব ডিস্কে ভাগ, কোনো অতিরিক্ততা নেই	100 % = N × আকার (2)	0 — একটি গেলেই সব শেষ
1 — মিররিং	প্রতিটি ডিস্কে হুবহু অনুলিপি	একটি ডিস্কের সমান (2)	N − 1
5 — বিতরণ প্যারিটি ★	প্যারিটি চক্রাকারে ছড়ানো	(N − 1) × আকার (3)	1
6 — দ্বৈ-প্যারিটি	RAID 5 + দুটি প্যারিটি ব্লক	(N − 2) × আকার (4)	2
10 — 1 + 0	জোড়া মিরর করে, তারপর সেগুলোতে স্ট্রাইপ	মোটের 50 % (4)	প্রতি জোড়ায় ১টি

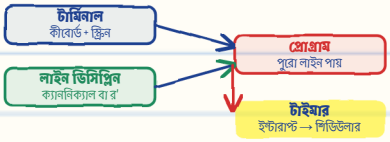
সমাধানকৃত উদাহরণ ✍ RAID হিসাব (ক) 4 × 1 TB-এর RAID 0 → **4 TB**, অতিরিক্ততা নেই। (খ) 4 × 1 TB-এর RAID 1 → **1 TB**, যেকোনো **3টি** বাদ যেতে পারে। (গ) 4 × 1 TB-এর RAID 5 → (4 − 1) = **3 TB**, ১টি ডিস্ক বাদ যায়, তবে পুনর্গঠনে সব লেখা হয়। (ঘ) 5 × 1 TB-এর RAID 6 → (5 − 2) = **3 TB** ও **2টি** বাদ যায়; 8 × 1 TB-এর RAID 10 → 8 ÷ 2 = **4 TB**। (ঙ) RAID 5-এ **ছোট** একটি লেখা পাড়ে **৪টি ডিস্ক I/O** — পুরোনো ডেটা পড়া, পুরোনো প্যারিটি পড়া, নতুন ডেটা লেখা, নতুন প্যারিটি লেখা।

মনে রাখার কৌশল "**শূন্যে স্ট্রাইপ, একে মিরর, পাঁচে এক প্যারিটি, ছয়ে দুই, দশে অর্ধেক।**" মিরর করে স্ট্রাইপ লেখা হয় **10**; স্ট্রাইপ করে মিরর লেখা হয় **01** — সংখ্যাগুলো কাজের ক্রমই বোঝায়।

SSD বনাম ম্যাগনেটিক ডিস্ক নেই হেড, নেই ঘোরা, যেকোনো অ্যাক্সেস ≈ 0.1 ms বিপরীতে 4–10 ms; পড়া ও লেখা সমান; **ওয়েয়ার লেভেলিং** লেখা ছড়িয়ে দেয়; TRIM মৃত ব্লক জানায়। প্রতি TB-এ এখনো দামি।

৮ টার্মিনাল, লাইন ডিসিপ্লিন, ক্লক ও টাইমার ইন্টারাপ্ট শেষ বিষয় ★

সংজ্ঞা **টার্মিনাল** *(Terminal)* হলো সেই ডিভাইস যার মাধ্যমে মানুষ কম্পিউটারের সঙ্গে কথা বলে — ইনপুটে কীবোর্ড, আউটপুটে ডিসপ্লে, একটি একটি অক্ষরে। OS কীবোর্ড সরাসরি প্রোগ্রামের সঙ্গে জোড়া দেয় না: মাঝখানে ডিভাইস-ইন্ডিপেন্ডেন্ট স্তরের একটি **লাইন ডিসিপ্লিন** বসে এবং বাফারিং, ইকো ও বিশেষ কী ঠিক করে দেয়।



টার্মিনাল আর কোনো ক্যারেক্টার ডিভাইসের মতোই – চারপাশে দুই স্তরের সফটওয়্যার

টার্মিনাল	এটা আসলে কী
ক্যারেক্টার টার্মিনাল	কীবোর্ড + অক্ষরের ডিসপ্লে; ইনপুট একটি **বাইট স্ট্রিম**, আউটপুট লেখা — একটি **ক্যারেক্টার ডিভাইস**
গ্রাফিক্স টার্মিনাল	**এস্কেপ সিকোয়েন্স** নেয় এবং পিক্সেল পর্যায়ে আঁকে
লাইন ডিসিপ্লিন	**ক্যাননিক্যাল** পুরো লাইন জোড়া করে ইকো করে; **র' *(raw)*** প্রতিটি কী চাপেই পাঠায় — এডিটর, গেম
সিউডো-টার্মিনাল	*SSH* যা ব্যবহার করে: একটি ফাইলে টার্মিনাল প্রোগ্রাম লেখে, লাইন ডিসিপ্লিন পড়ে, তারপর নেটওয়ার্কে পাঠায়

পরীক্ষার ফাঁদ টার্মিনাল কোনো "দূর কনসোল" নয়: এটি একটি *I/O* ডিভাইস, তাই এর **ড্রাইভার**, **লাইন ডিসিপ্লিন** ও **ইন্টারাপ্ট-চালিত ইনপুট** — তিনটিই এই লেসনের আগের পাতায়।

ক্লকের তিনটি কাজ — এবং প্রতিটি ছাড়া কী ভাঙে

১ · কেটে যাওয়া সময় মাপা। চলতে থাকা একটি কাউন্টার, প্রয়োজনে পড়া হয়

২ · সময়সীমা আরোপণ। **ইন্টারভ্যাল টাইমার** — শিডিউলিংকে প্রিম্পটিভ করে

৩ · CPU সময়ের হিসাব। প্রতিটি প্রসেসের টিকের হিসাব রাখা

টাইম-অফ-ডে ক্লক: দেয়াল-ক্লকের সময় রাখে — ব্যাটারিব্যাকড *CMOS/RTC*

হার্ডওয়্যার খুব সরল: প্রতি ক্লক টিকে একটি কাউন্টার এক কমে, শূন্যে পৌঁছালে সে **ইন্টারাপ্ট তোলে** এবং পরের মানে রিলোড হয়। ওই একটি ইন্টারাপ্টই একটি চলতে থাকা প্রসেসকে সারাবার *CPU* আটকে রাখতে দেয় না। সাধারণ টিক ***10 ms***, তাই আটটি প্রসেস একটি কোয়ান্টামে *80 ms*-এর রাউন্ডে *10 ms* করে পায়।

ক্লক বনাম টাইমার **ক্লক সময়** রাখে; **টাইমার গোনে** এবং ইন্টারাপ্ট তোলে। টাইম-অফ-ডে ক্লক পড়া যায় কিন্তু কখনো ইন্টারাপ্ট দেয় না; ইন্টারভ্যাল টাইমার সময়সূচি অনুযায়ী ইন্টারাপ্ট দেয়। আসল হার্ডওয়্যার দুটোকে একটি কাউন্টার ও একটি তুলনা রেজিস্টারে মিলিয়ে দেয়।

মনে রাখার কৌশল **"ক্লক সময় বলে, টাইমার প্রসেসকে থামতে বলে।"** চলমান প্রসেসকে *CPU* থেকে সরাতে পারে শুধু একটি ইন্টারাপ্ট, আর সেই ইন্টারাপ্ট সবসময় আসে টাইমার থেকে।

পরীক্ষার ফাঁদ *(i)* টাইমার ইন্টারাপ্ট ছাড়া শিডিউলিং **নন-প্রিম্পটিভ** হয়। *(ii)* কোয়ান্টাম আরোপণ করে টাইম-অফ-ডে ক্লক নয়, ইন্টারভ্যাল টাইমার; লিনাক্স এখন **টিকলেস** চলে।

রাতের পাতা

তথ্য-পত্র

কন্ট্রোলার = মধ্যস্থামী
স্ট্যাটাস বলে **বিজি / রেডি / এরর**
পোর্ট-ম্যাপড = IN/OUT, নিজস্ব স্পেস
ট্র্যাপ = ভেতরের এরর
পোলিং = CPU ঘোরে
DMA = কন্ট্রোলার, পুরো ব্লক
সাইকেল চুরি · বাস্টঁ আঁকড়ে · চুপচাপ অপেক্ষা
I/O স্তর: লাইব্রেরি, সাধারণ, ড্রাইভার, হ্যান্ডলার
I/O নীতি: FCFS, প্রায়োরিটি, SSTF, EDF
বাফার = গতি ও আকারের অমিল
স্পুল = একান্ত ডিভাইস ভাগ
ডাবল বাফারে $T = max(C,M)$
$T_r = 30 \div rpm$ সেকেন্ড
অ্যাক্সেস = সিক + স্পিন + শিপ
SSTF ভোকায় · SCAN হলো লিফট
LOOK থামে শেষ অনুরোধে
বুট ব্লক = ব্লক 0 = বুট লোডার
RAID 5 ছোট লেখা = **৪টি** I/O
লাইন ডিসিপ্লিন: ক্যাননিক্যাল বা র'

কন্ট্রোল, স্ট্যাটাস, ডেটা রেজিস্টার
ভেক্টর টেবিল = হ্যান্ডলারের ঠিকানা
মেমোরি-ম্যাপড = লোড/স্টোর, **স্পেস খায়**
ইন্টারাপ্ট = ডিভাইস, অ্যাসিঙ্ক্রোনাস
ইন্টারাপ্ট = CPU প্রতি বাইটে
DMA = প্রতি ব্লকে **১** ইন্টারাপ্ট
ব্লক ডিভাইস = ব্লক · ক্যারেক্টার = স্ট্রিম
ডিভাইস কে জানে শুধু ড্রাইভার
SSD-তে **র‍্যান্ডম সেরা**
ক্যাশ = পুনরায় পড়া এড়ায়
স্পুলিংয়ে লাগে একটি **ডিমন**
প্লাটার → ট্র্যাক → সেক্টর → সিলিন্ডার
T_t = বাইট ÷ রেট
প্রতি ট্র্যাকে সমান সেক্টর: **না** (ZBR)
C-SCAN = সারি, একদিকে
ফরম্যাটিং: গ্যাপ · ECC · স্পেয়ারিং
RAID 0/1/5/6/10: 0 · 1 · N−1 · N−2 · N/2
SSD ≈ 0.1 ms · ডিস্ক ≈ 8 ms
ক্লক সময় রাখে · টাইমার ইন্টারাপ্ট দেয়

→ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. CPU একটি ডিভাইসকে দেখে **তিনটি রেজিস্টার**
2. মেমোরি-ম্যাপড I/O-র খরচ **অ্যাড্রেস স্পেস**
3. ইন্টারাপ্ট-চালিত I/O = **প্রতি বাইটে ১টি** ইন্টারাপ্ট
4. DMA = প্রতি ব্লকে **একটি** ইন্টারাপ্ট
5. 7200 rpm ⇒ গড় রোটেশনাল ডিলে **4.17 ms**
6. 7200 rpm + 4 ms সিক + 4 KiB ⇒ **8.19 ms**
7. একই ব্যাসার্ধের সব ট্র্যাক = সিলিন্ডার
8. এই সারিটি SSTF-তে 53 থেকে ⇒ **236**
9. একই সারি SCAN-এ ⇒ **331**; LOOK-এ ⇒ **299**
10. 4 ডিস্কের RAID 5 ⇒ **3** ব্যবহারযোগ্য, **1** বাদ যায়
11. কীবোর্ড, মাউস, প্রিন্টার = **ক্যারেক্টার** ডিভাইস
12. শুধু **টাইমার ইন্টারাপ্ট** প্রসেসকে প্রিম্পট করতে পারে

সবচেয়ে বড় ফাঁদগুলো পোলিংকে কারণে "ইন্টারাপ্ট লাগে না, তাই দক্ষ" বলা · ইন্টারাপ্ট-চালিত I/O সবসময় পোলিং-এর চেয়ে ভালো বলা · DMA-কে CPU সরিয়ে দেওয়া বলা · প্রতিটি ট্র্যাকে সমান সেক্টর বলা · সিলিন্ডারকে আকৃতি ভাবা · SCAN শেষ অনুরোধে থামে বলা · C-SCAN ফেরার পথে সামলায় বলা · **RAID 5 দুটি ব্যর্থতা সহ্য করে** বলা · RAID 10 কে 01 লেখা · লো-লেভেল ফরম্যাটিংকে ব্যবহারকারীর কাজ বলা · টাইম-অফ-ডে ক্লককে কোয়ান্টাম আরোপণকারী বলা।

সব কৌশল একসাথে "তিনটি রেজিস্টার, একটি টেবিল" · "ধীরে ঘোরো, দ্রুতে ইন্টারাপ্ট, আরও দ্রুতে DMA" · "সাইকেল চুরি, বাস্টঁ আঁকড়ে, চুপচাপ অপেক্ষা" · "লাইব্রেরি, সাধারণ, ড্রাইভার, হ্যান্ডলার" · "বাফার ফাঁক বাঁধে, স্পুল ভাগ করে, ক্যাশ পুনরায় পড়া এড়ায়" · "সিক, স্পিন, শিপ" · "30 ভাগ rpm" · "SSTF কাছের, SCAN লিফট, C-SCAN সারি" · "শূন্যে স্ট্রাইপ, একে মিরর, পাঁচে এক, ছয়ে দুই, দশে অর্ধেক" · "ক্লক সময় বলে, টাইমার প্রসেসকে থামতে বলে"।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. কন্ট্রোলার পুরো ব্লকটা সরাসরি মেমোরিতে নিয়ে গিয়ে শেষে CPU-কে ইন্টারাপ্ট দেয়। প্রযুক্তিটি কোনটি:

ক) প্রোগ্রামড I/O খ) পোলিং
গ) ডাইরেক্ট মেমোরি অ্যাক্সেস ঘ) স্পুলিং

২. পোর্ট-ম্যাপড I/O-র তুলনায় মেমোরি-ম্যাপড I/O-র প্রধান খরচ কী:

ক) বিশেষ ইনস্ট্রাকশন লাগে খ) ইন্টারাপ্ট তুলতে পারে না
গ) DMA বাধ্য ঘ) মেমোরি অ্যাড্রেস স্পেস খায়

৩. কম গতিতে একটি হ্যান্ডলার ট্রান্সফারের চেয়ে বেশি খরচ হতে পারে, তাই কোন পদ্ধতি **দ্রুত** হতে পারে:

ক) DMA খ) পোলিং
গ) সাইকেল স্টিলিং ঘ) স্পুলিং

৪. প্রতিটি প্লাটারের একই ব্যাসার্ধের ট্র্যাকগুলো মিলে কী:

ক) সিলিন্ডার খ) সেক্টর
গ) সারফেস ঘ) স্পিন্ডল

৫. 7200 rpm ডিস্কের গড় রোটেশনাল ডিলেই কত:

ক) 8.33 ms খ) 2.08 ms
গ) 4.17 ms ঘ) 16.7 ms

৬. নিচের কোনটি **ক্যারেক্টার** ডিভাইস, ব্লক নয়:

ক) হার্ড ডিস্ক খ) SSD
গ) USB ফ্ল্যাশ ড্রাইভ ঘ) কীবোর্ড

৭. কোন ডিস্ক নীতিতে কোনো অনুরোধ একটি পূর্ণ স্ক্যানের বেশি অপেক্ষা করে না:

ক) FCFS খ) SCAN
গ) SSTF ঘ) র‍্যান্ডম

৮. হেড 53-এ, সারি 98, 183, 37, 122, 14, 124, 65, 67। SSTF-তে মোট হেড চলাচল:

ক) 236 খ) 640
গ) 331 ঘ) 299

৯. প্রয়োজনীয় ট্র্যাকে হেড সরানোকে বলে:

ক) রোটেশনাল ডিলে খ) ট্রান্সফার টাইম
গ) কিউইং ডিলে ঘ) সিক টাইম

১০. টাইম-অফ-ডে ক্লক এমন কী করে যা ইন্টারভ্যাল টাইমার করে না:

ক) সময়সূচি অনুযায়ী ইন্টারাপ্ট তোলে খ) দেয়াল-ক্লকের সময় রাখে
গ) কোয়ান্টাম শেষ করে ঘ) প্রসেসের CPU সময়ের হিসাব রাখে

উত্তরমালা → ১-গ · ২-ঘ · ৩-খ · ৪-ক · ৫-গ · ৬-ঘ · ৭-খ · ৮-ক · ৯-ঘ · ১০-খ

লিখিত প্রশ্ন অপারেটিং সিস্টেমে I/O সফটওয়্যারের স্তরগুলো ব্যাখ্যা করো। ডাইরেক্ট মেমোরি অ্যাক্সেস ও এর বাস ব্যবহারের তিনটি পদ্ধতি লেখো। ১০ নম্বর

১ **চারটি স্তর** — (১) **ইউজার-লেভেল I/O সফটওয়্যার**: প্রোগ্রাম যে লাইব্রেরি বলে, আউটপুট লাইনে বাফার করে। (২) **ডিভাইস-ইন্ডিপেন্ডেন্ট OS সফটওয়্যার**: সমান নাম, ডিভাইস বরাদ্দ, বাফারিং, ব্লকের আকার, ত্রুটির রিপোর্ট। (৩) **ডিভাইস ড্রাইভার**: একমাত্র হার্ডওয়্যার-সচেতন স্তর, সাধারণ অনুরোধকে কন্ট্রোলার কমান্ডে বদলায়। (৪) **ইন্টারাপ্ট হ্যান্ডলার**: ইন্টারাপ্টে চলে, কনটেক্সট সংরক্ষণ করে, ডেটা সরায়, অপেক্ষায় থাকা প্রসেস জাগায়। ৪

২ **DMA** — নিজে কন্ট্রোলার ডিভাইস ও মেইন মেমোরির মধ্যে **পুরো ব্লক** সরায়; CPU কেবল মেমরির ঠিকানা, গণনা ও ডিভাইস পোর্ট সাজায়, তারপর অন্য কাজ করে এবং **ব্লক সরে গেলে একবার** ইন্টারাপ্ট পায়। ৩

৩ **তিনটি পদ্ধতি — সাইকেল স্টিলিং**: একবারে এক মেমোরি সাইকেল নিয়ে বাস ফেরত দেয়, তাই CPU ধীর হলেও থামে না। **বার্স্ট মোড**: কন্ট্রোলারের নিজস্ব FIFO থেকে পুরো ব্লকের জন্য বাস ধরে রাখে, থ্রুপুট সবচেয়ে বেশি। **ট্রান্সপারেন্ট মোড**: CPU খালি থাকলেই বাস ব্যবহার করে। ৩

→ পরীক্ষার আগে ✓

- ✓ I/O স্তরের চারটি স্তরের নাম বলতে পারি
- ✓ CPU খরচ দেখে পোলিং, ইন্টারাপ্ট ও DMA আলাদা করতে পারি
- ✓ rpm, সিক ও রেট থেকে গড় অ্যাক্সেস সময় বের করতে পারি
- ✓ SCAN-এ অনুরোধের সারি ট্রেস করে মোট চলাচল বের করতে পারি
- ✓ RAID-এর আয়তনের হিসাব করে বলতে পারি কতটি ডিস্ক বাদ যাবে
- ✓ ব্লক ও টাইমারের পার্থক্য বলতে পারি